BMS-API + MCP-server — specificatie voor IT

Opgesteld 2026-10-19, in opdracht van Brent Calaerts (Blitz Power). Bedoeld om door te sturen naar IT zodat zij weten wat ze moeten bouwen.

Kort samengevat (voor Brent)

Dit document vraagt IT om **twee dingen** te bouwen bovenop jullie beheersysteem (hier "BMS" genoemd):

1. **Een leesbare API** — een afgeschermd "loket" waarmee software op een veilige manier gegevens uit BMS kan **ophalen** (niet wijzigen): laadpaal-logs en foutmeldingen, laadsessies/verbruik, paal- en statusinfo, en klant-/ticketinfo.
2. **Een MCP-server** bovenop die API — dat is een standaard-koppelstuk waarmee een AI-assistent zoals **Claude rechtstreeks die gegevens kan opvragen**. Zo moeten wij zelf niets bouwen: Claude praat met de MCP-server, de MCP-server praat met de API, de API praat met BMS.

Het doel: dat Claude bij een serviceticket **zelf de juiste logs en info kan ophalen** (voor triage en loganalyse), in plaats van dat jij alles manueel moet opzoeken en downloaden.

Wat jouw aandacht vraagt:

- Het blijft **alleen-lezen** — de API mag niets in BMS veranderen. Belangrijk, want zo kan er niets stuk.
- Er zitten **persoonsgegevens** in (klanten). IT moet dat GDPR-correct en afgeschermd houden.
- IT vult zelf het BMS-deel in (welk systeem, welke velden) — wij beschrijven enkel **wat** we nodig hebben, niet hoe hun systeem werkt.

Het technische deel hieronder is voor IT.

---

1. Doel & context

Blitz Power behandelt servicetickets rond laadpalen. Vandaag worden laadpaal-logbundels (charX) en bijhorende info **manueel** opgezocht en gedownload. We willen dit automatiseren: een AI-assistent (Claude) moet tijdens de triage van een ticket **zelf** de relevante logs en gegevens kunnen ophalen.

Daarvoor is een **programmeerbare toegang (API)** tot BMS nodig, en een **MCP-server** die deze toegang op een AI-vriendelijke manier aanbiedt.

Deze API staat **los van** de planningsapp; het is een aparte, herbruikbare dienst.

---

2. Wat we vragen (overzicht)

#	Component	Korte omschrijving
A	**Read-only REST API** op BMS	HTTPS-API die gegevens uit BMS teruggeeft als JSON. Geen schrijfacties.
B	**MCP-server** op die API	Een Model Context Protocol-server die de API-functies als "tools" aanbiedt aan AI-assistenten (Claude).
C	**Documentatie**	OpenAPI/Swagger voor de API, en de connectie-URL + auth-gegevens voor de MCP-server.

Hosting: bij voorkeur in de bestaande **Azure-omgeving van Blitz Power** (abonnement "Azure Blitz Power - BMS"), EU-regio (West Europe).

---

3. Scope

Binnen scope (alleen lezen):

Domein	Wat we willen kunnen ophalen
Laadpaal-/charX-logs & foutmeldingen	Logbundels (charX `.tar.gz`) per paal en periode ophalen/downloaden; lijst van fouten/alarmen/ERROR-vectoren; CP-toestanden.
Laadsessies & verbruik	Sessies per paal/periode/klant; sessiedetail (kWh, duur, start/stop, connector, resultaat).

Domein	Wat we willen kunnen ophalen
Paal-/stationinfo & status	Lijst van palen/stations; detail (type, firmware, locatie); actuele **online/offline-status**.
Klant-/contract-/ticketinfo	Opzoeken welke klant/contract bij een paal of sessie hoort; koppeling paal <-> klant; ticket-referenties.

Buiten scope (voorlopig):

- Alles wat **schrijft** of **stuurt** (palen herstarten, instellingen wijzigen, sessies starten/stoppen). Expliciet uitgesloten.
- Facturatie/betalingen.

---

4. De API — functionele vereisten (deel A)

IT mapt deze functies op de echte BMS-gegevens. De namen hieronder zijn voorstellen.

4.1 Endpoints (voorstel)

Endpoint	Geeft terug
`GET /v1/stations`	Lijst van palen/stations (met filter op status, locatie, type).
`GET /v1/stations/{id}`	Detail van één paal (type, firmware, locatie, klantkoppeling).
`GET /v1/stations/{id}/status`	Actuele status (online/offline, laatste contact, actieve sessie).
`GET /v1/stations/{id}/logs?van=&tot=`	Lijst beschikbare logbundels in een periode.
`GET /v1/logs/{logId}`	Download van één charX-logbundel (`.tar.gz`) of de inhoud ervan.
`GET /v1/stations/{id}/errors?van=&tot=`	Foutmeldingen/alarmen/ERROR-vectoren in een periode.
`GET /v1/sessions?station=&van=&tot=&klant=`	Lijst laadsessies met filters.
`GET /v1/sessions/{id}`	Sessiedetail (kWh, duur, start/stop, connector, resultaat).
`GET /v1/customers/{id}`	Klant-/contractinfo (minimaal, zie privacy).
`GET /v1/search?q=`	Vrij zoeken op paal, serienummer, klant of ticket (handig voor triage).

4.2 Algemene afspraken

- **Formaat:** JSON (logbundels als bestand/stream).
- **Tijd:** tijdstempels in ISO-8601 met tijdzone (of UTC), consistent.
- **ID's:** stabiele, unieke identificatoren per paal/sessie/klant.
- **Filtering & paginering:** op datum, station, klant, status; paginering op grote lijsten.
- **Fouten:** duidelijke HTTP-statuscodes + JSON-foutboodschap.
- **Versionering:** `/v1/` in het pad, zodat latere wijzigingen niets breken.
- **Documentatie:** een **OpenAPI/Swagger**-bestand aanleveren.

---

5. Niet-functionele & technische vereisten

Thema	Vereiste
Alleen-lezen	De API heeft enkel leesrechten op BMS (least privilege). Geen enkel schrijfpad.
Transport	Uitsluitend HTTPS.
Authenticatie	Machine-to-machine: **OAuth2 client-credentials via Microsoft Entra** (app-registratie) óf een API-sleutel per afnemer. Voorkeur: Entra/OAuth.
Autorisatie	Toegang beperkt tot geautoriseerde afnemers; eventueel scopes per domein (logs/sessies/klanten apart).
Hosting	In de Azure-omgeving van Blitz Power, EU-regio. Bv. Azure App Service, Container Apps of Functions.
Snelheidslimiet	Rate limiting om BMS te beschermen.

Thema	Vereiste
Logging/audit	Alle API-toegang gelogd (wie, wanneer, wat opgevraagd) — zeker voor de klantgegevens.
Privacy (GDPR)	Persoonsgegevens minimaal houden; enkel velden die nodig zijn; afgeschermd; bewaren/loggen conform GDPR.
Beschikbaarheid	Mag BMS niet belasten: eventueel via een kopie/cache of read-replica i.p.v. rechtstreeks op het productiesysteem.

---

6. De MCP-server (deel B) — zodat wij geen client hoeven te bouwen

6.1 Wat het is (voor IT)

Een **MCP-server** (Model Context Protocol, open standaard — modelcontextprotocol.io) stelt de API-functies beschikbaar als **"tools"** die een AI-assistent zoals Claude rechtstreeks kan aanroepen. Claude ontdekt de tools, roept ze aan met parameters, en krijgt het resultaat terug — zonder dat wij zelf integratiecode schrijven.

6.2 Vereisten

- **Type:** een **remote MCP-server** (HTTP/streamable transport), bereikbaar via een vaste HTTPS-URL, gehost in dezelfde Azure-omgeving.
- **Bouw:** met de officiële MCP-SDK (TypeScript of Python). De server is een dunne laag: elke tool roept intern de API (deel A) aan.
- **Alleen-lezen tools**, met duidelijke namen en beschrijvingen. Voorstel:

Tool	Doet	Parameters
`zoek_station`	Zoekt een paal op naam/serienummer/locatie	zoekterm
`station_info`	Geeft detail van een paal	station-id
`station_status`	Geeft actuele online/offline-status	station-id
`lijst_logs`	Lijst beschikbare logbundels	station-id, periode
`haal_log`	Haalt/ontsluit een charX-logbundel	log-id
`lijst_fouten`	Foutmeldingen/alarmen in een periode	station-id, periode
`lijst_sessies`	Laadsessies met filters	station/klant/periode
`sessie_detail`	Detail van één sessie	sessie-id
`zoek_klant`	Klant-/contractinfo opzoeken	zoekterm/id

- **Authenticatie van de MCP-server:** bij voorkeur **OAuth**, zodat hij als "connector" in Claude kan worden gekoppeld; een API-sleutel-header is een alternatief.
- **Oplevering:** de **connectie-URL** van de MCP-server + hoe hem te koppelen, plus de lijst met beschikbare tools.

---

7. Beveiliging & privacy (belangrijk)

- De API ontsluit **bedrijfs- en persoonsgegevens**. Daarom: alleen-lezen, geauthenticeerd, EU-gehost, toegang beperkt en gelogd.
- **Klantgegevens** (deel van domein 4): minimaliseren en afschermen; enkel wat voor triage nodig is.
- De credentials (OAuth-secret / API-sleutel) bewaren in **Azure Key Vault**, nooit los doorsturen.
- Toegang intrekbaar per afnemer.

---

8. Wat we van IT nodig hebben (checklist)

1. **Wat is BMS concreet?** Welk systeem/leverancier, waar draait het, wie beheert het?
2. **Hoe raken we aan de data?** Bestaat er al een API/database-toegang, of moet die eerst ontsloten worden?
3. **Datamodel:** welke velden bestaan er voor palen, sessies, logs, klanten — en hoe hangen ze samen (paal <-> klant <-> ticket)?

4. **charX-logs:** waar en hoe worden de logbundels bewaard, en kunnen ze per paal/periode opgehaald worden?
5. **Auth-aanpak:** Entra/OAuth of API-sleutel? Wie beheert de app-registratie?
6. **Belasting:** mag de API rechtstreeks op BMS, of werken we beter met een kopie/read-replica?
7. **Rechten in Azure:** wie mag de resources aanmaken (de planning-test liep hierop vast — zie aparte nota)?
8. **Tijdslijn & eigenaar:** wie bouwt en onderhoudt dit?

---

9. Aanbevolen fasering

1. **Fase 1 — lezen van het essentiële:** stations + status + charX-logs + foutmeldingen (dat dekt de ticket-triage grotendeels).
2. **Fase 2 — sessies & verbruik.**
3. **Fase 3 — klant-/contract-/ticketkoppeling** (met de privacy-afscherming).
4. **Fase 4 — MCP-server** bovenop wat er dan staat, zodat Claude het kan gebruiken.

Elke fase levert al meteen waarde: zodra fase 1 er is, kan Claude al logs en status ophalen bij een ticket.